

1. Saving Changes

These are your everyday commands for progressing your work.

Command	Path	Description
git add <file>	Work Dir $\rightarrow$ Index	Takes changes from your files and puts them in the staging area.
git commit	Index $\rightarrow$ HEAD	Takes the snapshot in the Index and saves it permanently in the HEAD.
git commit -a	Work Dir $\rightarrow$ HEAD	A shortcut that stages all tracked files and commits them in one go (skipping the manual add step).

2. Undoing & Restoring

These commands are used when you've made a mistake or want to revert to a previous state.

Command	Path	Description
git restore <file>	Index $\rightarrow$ Work Dir	Discards local changes by overwriting your file with the version currently in the Index.
git restore --staged <file>	HEAD $\rightarrow$ Index	"Unstages" a file. The changes stay in your Work Dir, but are removed from the Index.
git reset <file>	HEAD $\rightarrow$ Index	The older way to "unstage" a file (same as restore --staged).

git checkout <file>	Index $\rightarrow$ Work Dir	The legacy version of restore; overwrites your local file with the Index version.
---------------------	--	---

3. Checking Differences

To see what is different between these areas, use git diff.

- **git diff**: Compares **Work Dir** vs. **Index**. (What have I changed but not staged yet?)
- **git diff --cached**: Compares **Index** vs. **HEAD**. (What is staged and ready to be committed?)
- **git diff HEAD**: Compares **Work Dir** vs. **HEAD**. (What is the total difference since my last commit?)

4. Advanced: git reset

The reset command is unique because it can affect all three areas at once depending on the "flag" you use:

- **git reset --soft <commit>**: Moves **HEAD** only. Your Index and Work Dir stay exactly as they were.
- **git reset --mixed <commit>** (Default): Moves **HEAD** and updates the **Index** to match. Your Work Dir remains untouched.
- **git reset --hard <commit>**: Moves **HEAD**, updates the **Index**, and updates the **Work Dir**. This wipes out all uncommitted work to match the commit perfectly.

5. git stash

Sometimes you aren't ready to commit, but you need a clean slate.

- **git stash**: Takes everything in your **Work Dir** and **Index**, saves it in a secret "stack," and resets your environment to match the **HEAD**.
- **git stash pop**: Brings those changes back from the stack and reapplies them to your **Work Dir** and **Index**.